PERFORMANCE EVALUATION OF CONVOLUTION FILTERS USING OPENMP MULTITHREADING

Introduction

Image filtering with spatial convolution is a foundational task in computer vision and image editing. However, executing these operations sequentially scales poorly as image resolutions enter higher configurations. This project aims to build an independent image filtering engine using low-level C++ programming and to speed up its output using multithreading via OpenMP. The objective is to evaluate how effectively compiler can distribute uniform computational loads across hardware threads compared to a single-threaded execution.

What is convolution

In mathematics, Convolution is an operation that combines two functions to produce a third function, representing how the shape of one is modified by the other.

It is defined by the integral which involves flipping one function around the y-axis, shifting it by a variable amount , multiplying it with the second function, and integrating the product.

Here we define g as the kernel function.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

The four important steps in this process are –

Flipping

The kernel function is flipped around the vertical axis to become $g(\tau)$

Shifting

The flipped kernel is shifted dynamically along the horizontal axis by a variable displacement factor t

Multiplication

The shifted kernel is multiplied point by point with the stationary input function f

Integral

The resulting product is integrated across the entire domain

Using 2d spatial convolution for image processing

We apply the mathematical concept of convolution to image processing by using a stationary input image and then using a small localized matrix (typically of 3 x 3 or 5 x 5) known as kernel or convolution filter.

Convolution involves sliding a kernel over each pixel of the image and computing the "weighted sum" of the pixel's neighbourhood. The result is a new transformed image.

The 2D convolution equation utilized directly within this engine's function to apply filters is expressed as:

$$h(x,y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} f(x+i, y+j) * k(i,j)$$

$h(x,y)$ is the newly calculated, transformed pixel intensity stored in destination of final filtered image.

$f(x+i, y+j)$ represents the pixel intensity in the neighbourhood of the input image

$k(i,j)$ represents the coefficient at position (i,j) within the 3 x 3 kernel matrix

Filter Formulation

Different filters require different kernels to produce their respective effects. The filters I have decided to use for this project are explained here –

Gaussian Blur

Gaussian blur is primarily used to **reduce image noise** and **smooth out details** by applying a 2d gaussian distribution-based kernel where closer pixels have more influence than distant ones. This non-uniform filter effectively gets rid of high-frequency noise, such as grains, while preserving edges better than simple mean/box blur filters.

The kernel for gaussian blur is $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$

Sharpen

Sharpening image filters are primarily used to **enhance spatial resolution** by highlighting fine details, restoring blurred information, and increasing the contrast at object boundaries. This process makes images appear clearer and more defined.

The kernel for sharpen is $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

Sobel Edge Detection

The Sobel edge detection filter is primarily used to identify object boundaries and structural properties by convolving the image with two 3x3 kernels to approximate the horizontal (Gx) and vertical (Gy) derivatives, allowing for the detection of edges where there are strong changes in pixel intensity.

The kernel for Left Sobel is $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

The kernel for Top Sobel is $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$

Emboss

The emboss filter is a digital image processing technique used to create a three-dimensional (3D) or raised effect on flat images. It works by applying a convolution matrix to highlight edges and contours, shifting brightness values to generate an illusion of depth and shadow.

The kernel for Emboss is $\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$

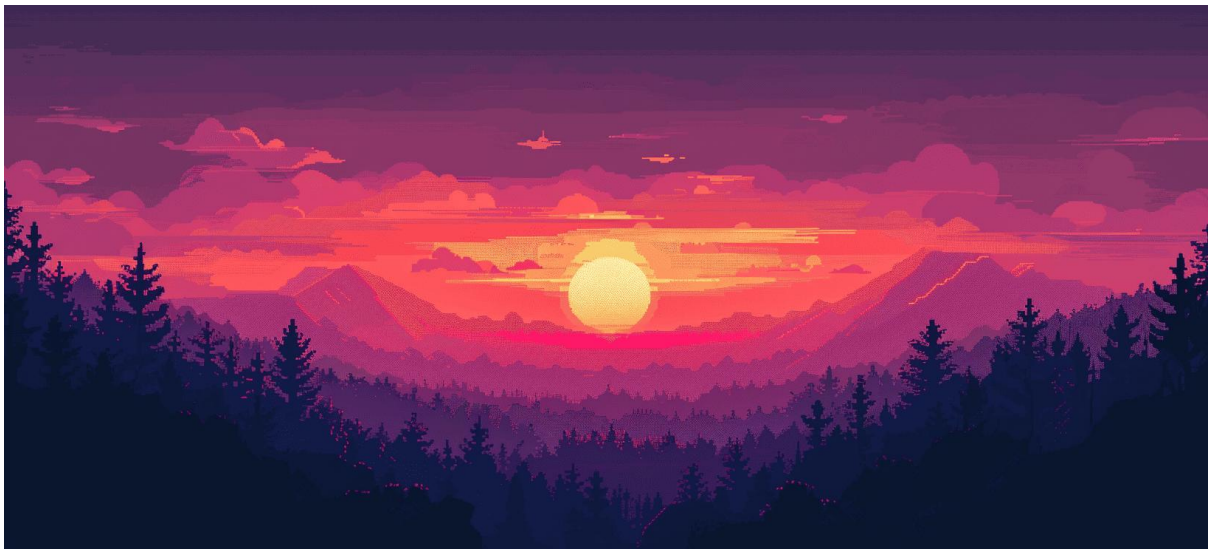
Architecture of the system

The system is constructed entirely in C++ using the following –

- Stb_image.h and stb_image_write.h to handle image I/O
- Input images are passed as continuous single dimensional array of unsigned characters where index sequences track the color channels (R,G,B)
- OpenMP is used to enable multi-threading so that I can compare time taken for processes with their serial counter-part. A single master thread executes sequentially until it encounters a parallel directive (#pragma omp parallel). It then forks a team of worker threads to execute the marked code block concurrently

Environment used for the engine and test image

- 8 Threads using the *omp_set_num_threads*(8) command. I am not using the omp_get_max_threads() tag because due to key **Amdahl's law** in multi-threading that states speedup is fundamentally limited by the sequential (non-parallelizable) fraction of a program no matter how many threads you add, the serial portion creates a threshold
- g++ compiler with -O3 tag for high level of optimization and -fopenmp tag to enable usage of OpenMP for multi-threading
- The following image of 1632 x 736 resolution was used as a test image



Kernel Function Overview

In the code I have actually used an applyKernel function that uses 4 loops to perform the 2d convolution with time complexity of $O(n^2)$ which is shown here

It takes every pixel in an image and computes a weighted sum of its surrounding 3×3 neighbourhood, producing a filtered output image. Depending on the kernel values passed in, this can perform operations like blurring, sharpening, edge detection, embossing, etc.

#pragma omp parallel for schedule(static) — Parallelizes the outer loop across CPU threads using OpenMP, dividing rows among threads for performance.

Outer loops (y, x) — Iterates over every pixel, skipping the border (starts at 1, ends at dimension − 1) to avoid border condition.

Inner loops (KernelY, KernelX) — Iterates over the 3×3 neighbourhood around the current pixel (offsets −1 to +1).

```
void applyKernel(unsigned char *img, unsigned char *result, int width, int height, int channels, float Kernel[3][3])
{
#pragma omp parallel for schedule(static)

    for (int y = 1; y < height - 1; y++)
    {
        for (int x = 1; x < width - 1; x++)
        {
            float r = 0, g = 0, b = 0;

            for (int KernelY = -1; KernelY <= 1; KernelY++)
            {
                for (int KernelX = -1; KernelX <= 1; KernelX++)
                {
                    int neighbourX = x + KernelX;
                    int neighbourY = y + KernelY;

                    int index = (neighbourY * width + neighbourX) * channels;

                    float KernelIndex = Kernel[KernelY + 1][KernelX + 1];

                    r += img[index] * KernelIndex;
                    g += img[index + 1] * KernelIndex;
                    b += img[index + 2] * KernelIndex;
                }
            }

            int outIndex = (y * width + x) * channels;

            result[outIndex] = (unsigned char)std::min(std::max((int)abs(r), 0), 255);
            result[outIndex + 1] = (unsigned char)std::min(std::max((int)abs(g), 0), 255);
            result[outIndex + 2] = (unsigned char)std::min(std::max((int)abs(b), 0), 255);
        }

    }
}
```

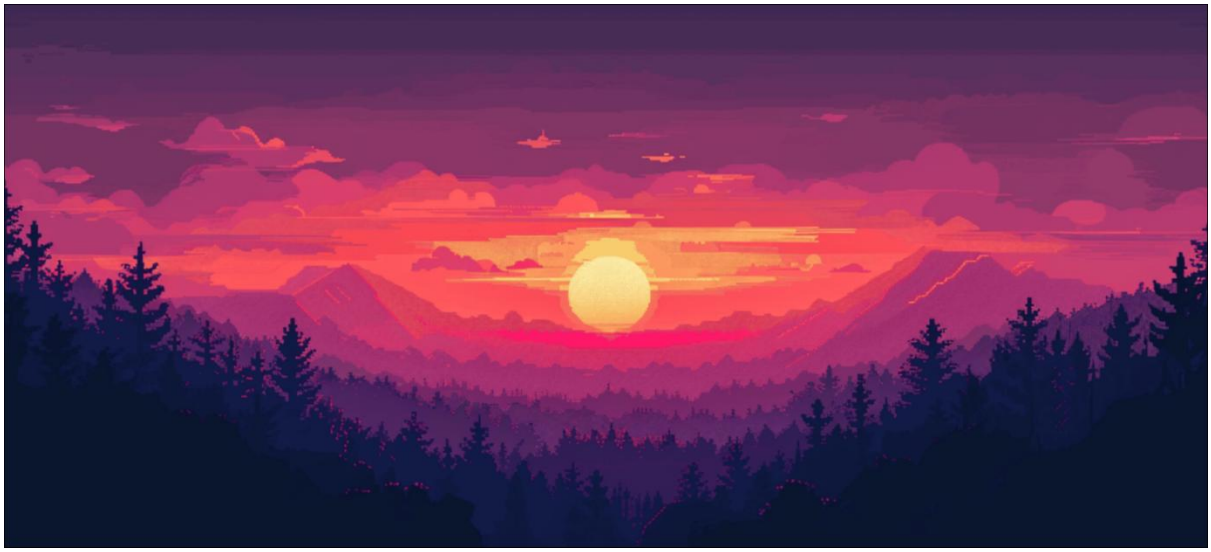
Performance Evaluation

The execution timing was captured by using the standard chrono library functions in C++.

Filter Module	Serial Execution	OpenMP Parallel	Speedup Factor
Grayscale	0.0017895 s	0.0011308 s	1.58251x
Gaussian Blur	0.0098732s	0.0017329s	5.6975x
Sharpen	0.0089714s	0.0020755s	4.32252x
Sobel Edge	0.0094078s	0.0033834s	2.78058x
Emboss	0.0082138s	0.0018886s	4.34915x

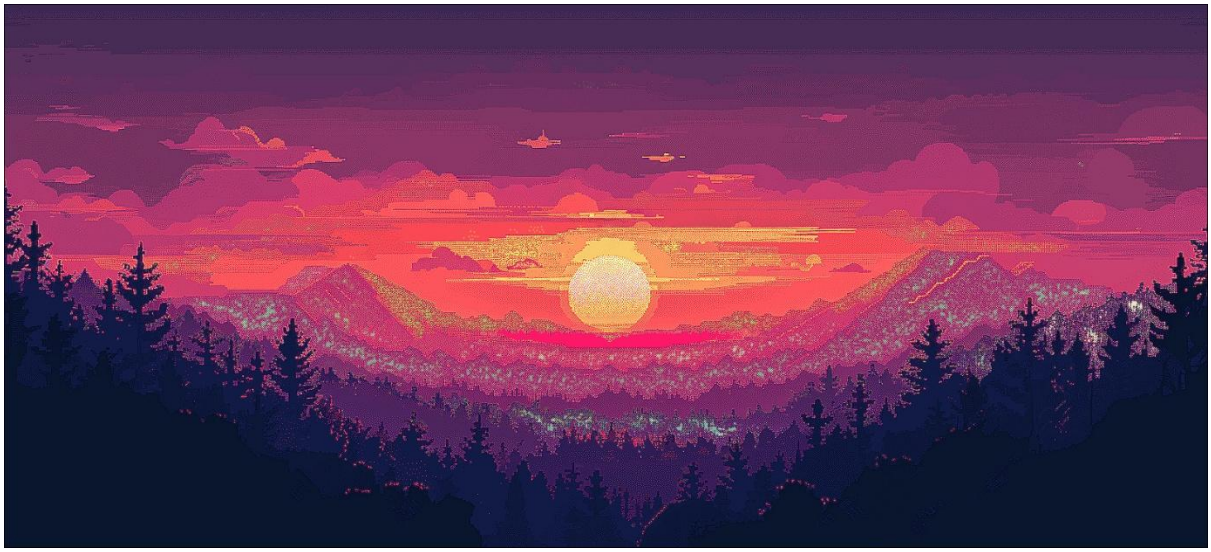
Result

Grayscale



Gaussian Blur

Emboss



Sharpen

Sobel Edge detection

Conclusion

This project successfully demonstrated the design and implementation of a C++ image filtering engine capable of applying spatial convolution filters, Gaussian Blur, Sharpen, Sobel Edge Detection, and Emboss to RGB images, and evaluated the performance improvement achieved through OpenMP multithreading.

The results confirm that parallelizing convolution workloads across hardware threads yields significant speedup, with Gaussian Blur achieving the highest gain of 5.69x due to its highly uniform per-pixel workload that distributes evenly across threads.

Filters with heavier per-pixel computation, such as Sobel Edge Detection, which requires two separate kernel passes and a square root operation per pixel, showed comparatively lower speedup at 2.78x, reflecting that a larger fraction of the work resists straightforward parallelization.

The Grayscale conversion showed the most modest gain of 1.58x, as it’s math is pretty simpler than the other filters meaning threads frequently stall waiting on memory access rather than performing arithmetic.

Overall, the experiment validates that convolution-based image processing is a naturally parallel problem since each output pixel is computed independently of others and that OpenMP's #pragma omp parallel is an effective and low-overhead approach to exploiting this.

Future Scope

The current engine is constrained to a fixed 3x3 matrix footprint. While highly effective for sharpen and emboss, advanced applications like heavy cinematic blurs require larger kernels (5 x 5 or 7 x 7).

Use hardware optimization (SIMD) to force the CPU to load multiple pixels into its registers at the exact same time.

Move the image loops off the CPU entirely and write them for a graphics card using NVIDIA CUDA.